Memory as a Learned Policy:
Budgeted Write Operations for Long-Horizon LLM Agents

Parth Kocheta
University of Maryland

May 2, 2026

Abstract

Long-running language-model agents accumulate context faster than any context window can hold, so they store some of it externally. Every published memory system—Mem0, Letta, Mastra, Zep, our own KGmem—decides what to commit using hand-written prompts and heuristics. We ask whether the same decisions can be made by a learned policy supervised by downstream answer correctness, the same signal already used to evaluate the system.

Memory becomes a budgeted decision problem. Under a hard retention budget K, a write policy emits a short sequence of typed operations (`create_entity`, `update_state`, `add_relation`, `mark_contradiction`, `forget`, and others) into a typed-transition knowledge graph; a fixed hybrid retriever fuses BM25, dense, and named-entity lookups for the read side. The technical obstacle is credit assignment: the policy emits four to twelve tokens of tool calls per turn, but the reward only arrives later, when a held-out question gets answered well or badly. Concurrent memory-RL work (Memory-R1, Mem-α) feeds GRPO a single trajectory-level scalar that diffuses uniformly over those tokens; per-op resolution is lost.

We propose *per-op counterfactual marginal utility*. For each mutating op a_i in a write trajectory, we replay the trajectory without it and measure how the held-out QA judge score changes. The op gets credit equal to the difference. The trajectory reward GRPO sees is the sum of these per-op deltas, but each op now carries its own signal at advantage time. Lookups and noops are skipped — their effect on the post-write graph is zero by construction — which keeps the cost at roughly 5× a trajectory-level signal in exchange for per-op resolution. The technique is the COMA cooperative-RL baseline applied at the granularity our typed op vocabulary makes natural; the closest concurrent memory-RL work that gives op-level rewards (DeltaMem) uses a state-distance signal, not an outcome-attribution one.

The success criterion is strict. A learned write policy must beat random subsampling at matched retention budget — the content-agnostic efficiency frontier — and a BM25 heuristic that picks turns by query overlap, on both LoCoMo and LongMemEval-S. [TBD: Headline numbers fill in once the Phase B run lands.]

Facts live in the store, strategy lives in the weights. The store holds whatever the user gives it; the strategy transfers without moving any of their data.

1 Introduction

The problem in one sentence. Long-running AI assistants need long-running memory; today every system that does this in production hand-codes the decision of what to write down. Mem0 prompts an LLM to choose between `ADD/UPDATE/DELETE/NOOP` on each new turn [8]. Letta moves items between three OS-style memory tiers via prompted function calls [17]. Zep maintains a knowledge graph whose edges expire under hand-written rules [19]. Mastra runs two background

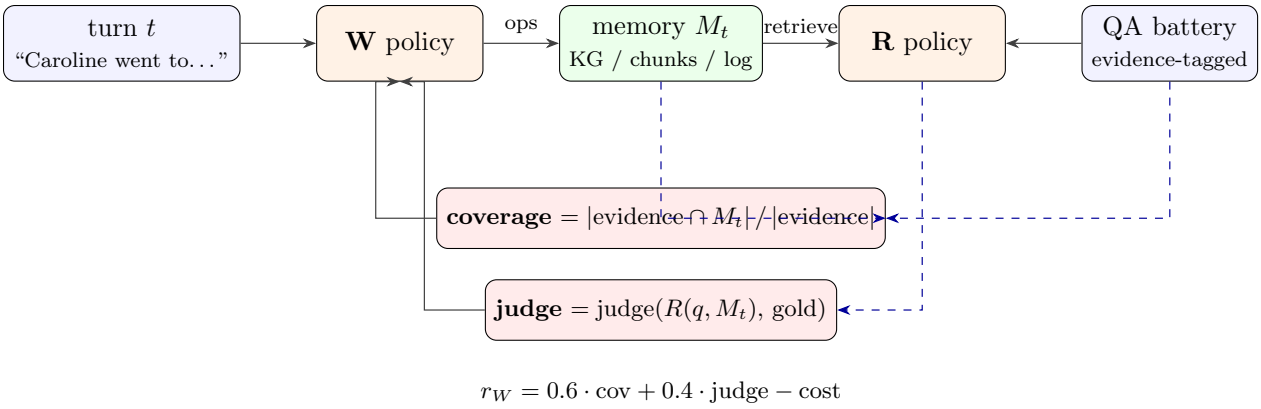

Figure 1: The write policy W ingests one conversation turn and emits memory ops; the read policy R later retrieves from the resulting store M_t to answer questions. W's reward blends a deterministic *evidence-coverage* term against LoCoMo's per-question dia_id labels with an LLM-judge term over R's answer. Coverage is dense and free; judge keeps the policy honest about whether the stored content is actually answerable.

LLM agents (*Observer* and *Reflector*) that compress raw turns into a bullet log on a fixed schedule and reach 94.87% on LongMemEval with GPT-5-mini [15]. KGmem, the prior-art typed-transition graph from the lead author that we use as one of our three backends, resolves new mentions against existing entities via a three-tier string/embedding/LLM cascade with thresholds chosen by inspection. Each of these works in practice. Each is also a place where the control logic is frozen — the system cannot improve from the same signal that was used to evaluate it.

Why hand-coding is fragile. A prompt cannot adapt to one user's writing style without being rewritten. A threshold cannot tell that a particular conversation has many proper nouns and few facts, or the reverse. A static rule cannot learn from its own mistakes: when Mem0 fails to extract a fact (the Mem0 paper itself reports a $\sim 40\%$ extraction-failure rate), the system has no way to attribute the failure or update the rule. The recent Memory-R1 work [2] answers this by training a memory manager with reinforcement learning end-to-end on QA accuracy. Mem-α [25] and DeltaMem [28] extend the same idea with richer memory ontologies and longer episode horizons. The direction is right; the open question is the shape of the reward that turns "the agent answered well" into a useful gradient on "which write op deserved the credit?"

The credit-assignment problem, made concrete. A write policy that ingests one conversation turn typically emits two to four tool calls — maybe a `lookup_entity` to check whether "Caroline" already exists, then a `create_entity` that adds her, then an `add_relation` that links her to the LGBTQ support group she mentioned, then a `noop` on the next turn that's just a greeting. After this, future questions get asked. The reader retrieves from whatever the write policy left behind and tries to answer. The judge returns one number per question. GRPO [22], the standard critic-free RL algorithm for this kind of training, takes the trajectory's reward and applies it as a uniform advantage to every emitted token. So the four-token `create_entity` call and the four-token `noop` get the same gradient signal, despite making opposite contributions to the answer. This is the diffuse-credit problem and concurrent memory-RL work runs into it directly: Memory-R1 uses final-answer EM as a single per-trajectory scalar; Mem-α pools accuracy across an entire dialogue.

Our solution. We propose *per-op counterfactual marginal utility*. For each mutating op a_i the policy emitted, we replay the trajectory without it, run the same reader against the counterfactual memory state, and ask the judge whether each held-out question still gets answered. The op's credit is the average drop in judge score it would have caused by being absent. Lookups and noops are skipped because their effect on the post-write graph is zero by construction; only the mutating ops get attribution. The trajectory reward GRPO sees is the sum of per-op deltas, but each op now contributes its own credit signal at advantage time. The technique is the COMA counterfactual baseline from cooperative multi-agent RL [9] applied at the op granularity our typed vocabulary makes natural. The closest concurrent memory-RL work that gives op-level rewards is DeltaMem [28], which uses a state-distance signal (Memory-Levenshtein); ours is the first outcome-attribution variant in this setting.

Why this matters. The framing has two payoffs. For research, it makes memory a *learned operating system* — strategy in the weights, facts in the store — with a backend-agnostic op vocabulary that compiles to three different storage substrates we test. For deployment, the same property gives a useful asymmetry: a single trained adapter can run against any user's local store, no per-user fine-tuning, no need to move user data into a shared corpus. The store holds whatever the user gives it; the strategy transfers without moving any of their data.

Contributions.

1. A reframing of LLM-agent memory as a *budgeted decision problem*: under a hard retention budget $K_{\max}$, the learnable layer is the write policy that picks which K items to commit. Read becomes a hand-coded hybrid retrieval stack that the write policy optimises against.
2. **Per-op counterfactual marginal utility** as the dense write reward for memory-RL GRPO. Adapted from the COMA cooperative multi-agent baseline; novel in the memory-write setting, where concurrent work uses either trajectory-level outcome rewards (Memory-R1, Mem-α) or state-distance per-op rewards (DeltaMem) that don't directly say what the downstream reader needs.
3. A backend-retriever-aligned evaluation protocol that controls for the dominant confound in memory-system comparison: a paired LoCoMo audit shows mismatched retrievers can swing QA accuracy by 0.40 in absolute terms (McNemar $p = 1.9 \times 10^{-6}$). We run training and evaluation on the same hybrid retriever (BM25 + dense + NER fused with RRF).
4. Two strict baselines the learned policy must beat: random subsampling at matched F (the content-agnostic efficiency frontier) and a BM25 heuristic that picks turns by query overlap. We report area under the efficiency curve (AUEC) as the headline metric.
5. Empirical results on LoCoMo and LongMemEval-S, plus a head-to-head against KGmem's hand-coded extraction on personal ChatGPT-export data with self-generated questions (Section 3.6 Mode C). [TBD: Numbers fill in once the long Phase B run completes.]

Figure 1 shows the data-flow once. The rest of the paper goes through the parts in order: related work in §2, the op vocabulary and reward in §3, benchmarks and baselines in §5.1, and a discussion of what we believe and what we don't in §7.

2 Related Work

The right way to navigate the memory-systems literature is along two orthogonal axes: where do the facts live, and where does the write control come from. Table 1 places each major system on

the grid; the cell we sit in (external store, learned ops) is also the smallest and the most recently populated.

Table 1: The memory-systems design grid. Rows are storage substrate; columns are how the write decisions are made. The published memory-RL cluster (right column) all use trajectory-level outcome rewards except DeltaMem (state-distance) and ours (per-op outcome attribution).

Substrate	**Hand-coded ops**	**Learned ops**
Flat KV / chunk index	Mem0 [8], naïve RAG	—
Tiered OS-style memory	Letta / MemGPT [17]	—
Knowledge graph	Zep / Graphiti [19], KGmem	**this work**, DeltaMem [28]
Observation / summary log	Mastra OM [15], Generative Agents [18]	Mem-α [25], Memory-R1 [2]
Model weights	ROME / MEMIT [16], per-user fine-tuning, Cartridges [20]	—
Implicit (test-time recurrence)	Recursive Language Models [26], Titans [6]	—

2.1 The hand-coded column

Most production systems live here. The pattern is uniform: a prompt written by a researcher decides what to commit, sometimes with heuristics that fire afterward. Mem0 and Letta cap retention by tier or recency; Zep maintains valid-time intervals on KG edges; Mastra runs an *Observer* pass to extract candidate facts and a *Reflector* pass to compress the trail. Mastra's 94.87% on LongMemEval with GPT-5-mini is the strongest published number, but the cost is paid in LLM calls at write time — the Reflector runs on every conversation chunk, regardless of whether the fact it extracts will ever be queried. Mem0's own paper reports a $\sim 40\%$ extraction-failure rate, which is the visible signature of the underlying issue: a hand-tuned prompt can't tell which extractions matter for the answers users will eventually ask. Generative Agents [18] run periodic reflection cycles that condense observations into higher-level beliefs by the same template. Recursive Language Models [26] sit in a different cell entirely — they do no write-time work and instead use a test-time outer loop to decide how deeply to decompress the input context per query. RLM with Gemini 3 Flash hits 89.8% on LongMemEval, a strong result that suggests *when to spend test-time compute on memory* may be as important as *what to commit*. We treat RLM as orthogonal: a trained mempol write policy could be paired with an RLM-style read loop, and the two contributions would compose.

2.2 The learned-ops column

Concurrent published work in this cell is small but growing. **Memory-R1** [2] trains a Memory Manager (ADD, UPDATE, DELETE, NOOP) and an Answer Agent jointly with PPO/GRPO on 152 QA pairs from LoCoMo, MSC, and LongMemEval; the reward is final QA accuracy at the trajectory level. **Mem-α** [25] uses a richer memory ontology (core, episodic, semantic) and reports 13× generalisation in horizon length (training at 30k tokens, testing at 400k). **DeltaMem** [28] is the closest to ours: it gives op-level rewards via a Memory-based Levenshtein Distance on the post-write state. The difference between DeltaMem's reward and ours is exactly the difference between asking "did the op change the memory in a structured way?" and "did the op change what the reader can answer?". Both are reasonable. Ours is more directly aligned with the downstream

task; theirs is cheaper to compute. The earlier non-RL precedents are **ACE** [5] (evolves text-form playbooks based on execution feedback, but the playbook is a single prompt) and **Voyager** [24] (maintains a Minecraft skill library, with the LLM deciding what to add by trial-and-error rather than RL).

2.3 The RL machinery we use

GRPO [22] is the critic-free simplification of PPO that has become the default for tool-use training. For each prompt, sample G rollouts; the advantage of rollout i is $(r_i - \bar{r})/\text{std}(r)$. The group acts as a zero-bias baseline. This works well when reward is verifiable (math, code, judge-graded QA) and is what we use. **Search-R1** [12] applies GRPO to multi-turn search-then-answer; we use the Tinker-cookbook reproduction of Search-R1 as the structural template for our environment. **Reflexion** [23] and verbal-RL approaches update natural-language self-reflections rather than weights; they are fast and cheap but plateau because the policy is a static prompt. **Curriculum learning** [7] appears here because we use it on the read side: LoCoMo's question categories give us a natural easy-to-hard ordering. **LoRA** [11] is the adapter form-factor; we use rank-32 on Qwen3-4B-Instruct [13], trained via Tinker.

2.4 Temporal awareness as a separate axis

A distinct cluster of recent work shows that LLM agents fail at *behaving in time* — not at computing date arithmetic, but at deciding when to act. **Temporally Blind** [4] introduces TicToc, a 76-scenario benchmark that scores agent decisions about when to refetch stale context; the best model alignment with human preferences is 65%. **Real-Time Deadlines** [21] shows GPT-5.1 closes 4% of deals in time-pressured negotiation without a clock-in-prompt and 32% with one. **Robotouille** [10] reports a 47% $\rightarrow$ 11% collapse for ReAct agents the moment planning has to interleave synchronous and asynchronous actions. These are benchmarks of *behaviour*, not benchmarks of date-arithmetic (Test of Time [1] and Time-R1 [3] cover the latter, with synthetic curricula and dedicated three-stage RL respectively). The TemporalBench we propose in §4 extends TicToc's single-axis scoring to six axes of behaviour and is positioned squarely in this cluster.

2.5 What's missing, and what we add

Across the grid, no published system combines (i) discrete interpretable memory ops, (ii) chosen by a policy trained with outcome supervision, (iii) where the supervision attributes credit *per op* rather than per trajectory or per state-distance. Memory-R1 and Mem-α have (i) and (ii) but not (iii); DeltaMem has (i), (ii), and a per-op signal but the signal is state-distance, not outcome-attribution. That is the gap we close.

3 Method

3.1 The op vocabulary

We define a unified action space $\mathcal{A} = \mathcal{A}_R \cup \mathcal{A}_W$. Each action is a JSON tool call with a fixed schema.

Read ops $\mathcal{A}_R$.

- `memory_search(query, k, source)`: Retrieve top-k chunks by lexical (BM25), semantic (dense), or hybrid (RRF [22]) match. The retrieval substrate is whichever Backend the env was built with.
- `memory_expand(seed_uids, k_per)`: Pull adjacent chunks of seeds (one-hop graph follow / sibling-turn lookup).
- `memory_filter(predicate, value)`: Restrict the current hit set by session, speaker, or time window.
- `memory_rerank(strategy, query)`: Re-order hits by dense similarity, recency, or session order.

The episode ends when the model emits a final answer formatted as "`Answer: ...`".

Write ops $\mathcal{A}_W$.

- `noop`: This turn is not memory-worthy. The default.
- `lookup_entity(query, type, top_k)`: Find existing entities by name. The policy uses this to avoid duplicates.
- `lookup_relation(source_uid, target_uid)`: Inspect existing graph edges.
- `create_entity(name, type, state)`: Add a new entity. Type is one of nine: `person`, `project`, `tool`, `organization`, `belief`, `decision`, `concept`, `period`, `event`, `goal`.
- `update_state(uid, new_state, transition_type, trigger_summary)`: Mutate an entity. Transition type is one of `update`, `contradiction`, `resolution`, `archival`.
- `merge_entities(canonical_uid, alias_uid)`: Collapse two entities, moving transitions and edges.
- `add_relation(source_uid, target_uid, rel_type, description)`: Add a typed graph edge.
- `mark_contradiction(uid, contradicting_state)`: Flag that the current turn conflicts with the entity's prior state, but do not pick a winner.
- `forget(uid, reason)`: Soft-delete (archive). Keeps the transition history but marks the entity inactive.

3.2 Background: KGmem, a typed-transition knowledge graph

One of the three backends we evaluate predates this paper. We refer to it throughout as *KGmem*; the open-source implementation is the PIE memory system from prior work by the lead author, retained in our repository under `pie/`. We use a renamed reference here for readability and to keep the role of the data structure separate from the role of any specific control layer.

KGmem represents the user's world as a graph of typed entities (one of `person`, `project`, `tool`, `organization`, `belief`, `decision`, `concept`, `period`, `event`, or `goal`). Each entity carries a history of typed state transitions: every transition records a (*from state*, *to state*) pair labelled with one of `creation`, `update`, `contradiction`, `resolution`, or `archival`. In particular, contradiction is a first-class transition rather than an overwrite—an unusual choice among open knowledge-graph memory systems—which lets both the conflicting and prior states coexist until a later transition resolves them. KGmem also supports per-entity importance scoring and typed relationship edges; this paper uses the entity-plus-transition substrate only.

KGmem matters here for two reasons. First, the typed transitions give the write policy a richer action vocabulary than "store" and "delete"—it can emit `contradiction` when a turn conflicts with an existing entity, leaving both states retrievable for the read policy. Second, entity uids let the read policy chain queries (`lookup_entity` → `transitions_for(uid)`) when a question requires multi-hop reasoning over a single entity's evolving state. We re-use the data structure; the hand-coded extraction prompts and three-tier resolver that ship with KGmem are replaced by the learned write policy.

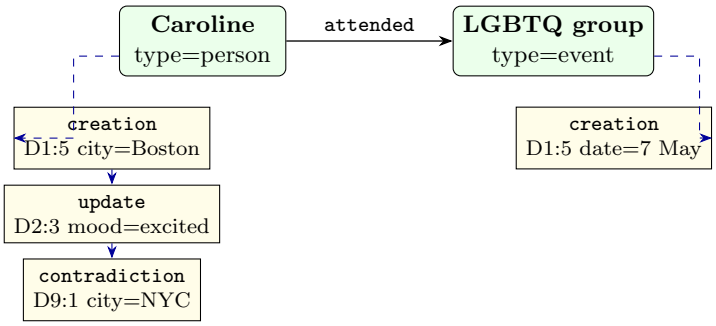

Figure 2: KGmem schema. Solid arrow: typed relationship between entities. Dashed arrows: typed state transitions, each tagged with the dia_id that caused it. Coverage reward sums dia_ids reachable through these provenance fields.

3.3 Backend abstraction

The op vocabulary is the invariant. Each backend is an adapter that compiles ops to native operations:

- **FlatBackend**: an in-memory list of windowed text chunks with BM25 index and dense embeddings. `memory_search` is BM25/dense/RRF [22]. Most write ops are no-ops; `create_entity` appends a chunk.
- **KGBackend**: wraps the KGmem WorldModel from Section 3.2—typed entities, typed transitions, and a relationship graph. Every write op compiles to a real KG mutation. Important: we re-use only the data structure—the write policy replaces KGmem's hardcoded extraction prompts and 3-tier string/embedding/LLM resolver entirely. The trained adapter learns what those heuristics did by hand, against outcome supervision.
- **MastraBackend**: an Observer/Reflector pipeline that compresses raw turns into a dated bullet observation log [15]. Read ops scan the log; write ops mostly no-op (the Observer/Reflector run as background processes).

The same trained policy runs on any of the three. Backend-transfer is one of our main empirical claims.

3.4 Episode definitions

Read episodes. One (question, gold_answer) pair per episode. The model emits read ops and a final answer. Reward:

$$r_R(\tau) = \text{judge}(\text{answer}, \text{gold}) - \lambda_R \, c_R(\tau)$$

where $c_R(\tau)$ is the number of tool calls (each priced at 0.005–0.01 reward units) and judge returns one of $\{0.0, 0.5, 1.0\}$ following the bucketing protocol of the LongMemEval paper. We use gpt-4o-mini as the judge during training (which correlates above 0.95 with gpt-4o on this style of free-form QA at roughly one fifth the cost), and switch to gpt-4o for the final held-out evaluation reported in Section 5.1.

Write episodes. One conversation turn per episode. The model sees the focal turn, two prior turns, and a top-k snapshot of pre-existing entities. It emits up to four write ops. The episode ends.

Hard retention budget. Before scoring, the post-W knowledge graph M_τ is pruned to at most $K_{\max}$ entities; lowest-importance entities are dropped first, with ties broken by recency of last update. We default to $K_{\max} = 12$ for per-turn episodes (calibrated so the budget binds without making the task trivially infeasible) and treat $K_{\max}$ as an ablation knob. Crucially, this means a policy cannot win the dense reward by "store every dia_id verbatim"—the budget makes that strategy infeasible by construction.

Reward, by example. Suppose a policy ingests one turn and emits four ops: $a_1 =$ `lookup_entity(''caroline'` $a_2 =$ `create_entity(''Caroline'', state={city: Boston})`, $a_3 =$ `noop`, $a_4 =$ `add_relation(Caroline, LGB` After the policy stops, four held-out questions about the next ten turns get asked, and the reader scores the resulting graph at 0.50 averaged across the four. We now ask: which op deserved the credit? We replay the trajectory three times, each time leaving one mutating op out, and rescore. Suppose the scores come back as 0.30 without a_2, 0.50 without a_3, and 0.40 without a_4. Then the per-op deltas are $\Delta_2 = 0.50 - 0.30 = +0.20$ (`create_entity` did real work), $\Delta_3 = 0$ (the noop didn't matter; we don't even compute it), and $\Delta_4 = +0.10$ (`add_relation` helped some). The trajectory-level reward GRPO sees is the sum, +0.30, but each op now contributes its own number at advantage time.

Reward, formally. Let $\tau = (a_1, \ldots, a_n)$ be the sequence of ops, $\mathcal{M}(\tau)$ the indices of mutating ops (everything except `lookup_*` and `noop`, which leave the post-W graph unchanged by construction), M_τ the post-budget memory state when all of τ is replayed, and $M_{\tau\setminus i}$ the state when τ is replayed without a_i. The per-op marginal utility is

$$\Delta_i(\tau, Q) \;=\; \frac{1}{|Q|}\sum_{q\in Q}\Big[\,\mathrm{judge}\big(R(q, M_\tau), \mathrm{gold}_q\big) - \mathrm{judge}\big(R(q, M_{\tau\setminus i}), \mathrm{gold}_q\big)\Big], \tag{1}$$

and the trajectory reward is

$$r_W(\tau) \;=\; w_{\mathrm{cf}}\sum_{i\in\mathcal{M}(\tau)}\Delta_i(\tau, Q) \;+\; w_{\mathrm{qa}}\cdot\frac{1}{|Q|}\sum_{q\in Q}\mathrm{judge}\big(R(q, M_\tau), \mathrm{gold}_q\big) \;-\; \lambda_W\, c_W(\tau). \tag{2}$$

The QA term anchors the policy to absolute downstream accuracy; the counterfactual term provides per-op attribution. We default to $w_{\mathrm{cf}} = 0.7$, $w_{\mathrm{qa}} = 0.3$, $\lambda_W = 0.001$, $K_{\max} = 12$. Op cost $c_W(\tau)$ is small because the hard retention budget does most of the regularising; we report it for diagnostic logging.[1]

[1]Earlier drafts used *evidence coverage* (fraction of a question's gold dia_ids preserved in M_τ) and *reader-overlap* (Jaccard of dia_ids the reader pulls from full text vs. those in the KG). Coverage correlates with the trivial turns-kept

Cost. The counterfactual evaluation runs $1 + |\mathcal{M}(\tau)|$ replays times $|Q|$ reader+judge calls per trajectory. With our typical $|\mathcal{M}| \approx 2$ and $|Q| = 4$ that's ~ 12 extra LLM calls per rollout, roughly $5\times$ the cost of a trajectory-level signal in exchange for per-op resolution. The replays themselves are local (no Tinker round-trips), parallelised via `asyncio.gather`, and reuse the full-state score as the w_{qa} anchor so we do not pay for it twice.

Connection to prior work. The technique is the COMA counterfactual-baseline trick [9] from cooperative multi-agent RL, applied at the op granularity. Concurrent memory-RL work uses different signals: Memory-R1 [2] and Mem-α [25] attribute reward at the trajectory level (final QA accuracy as one scalar), while DeltaMem [28] uses an op-level signal too but defines it as the Memory-Levenshtein distance between pre- and post-op states. State-distance asks "did the op change the memory in a structured way?"; outcome-attribution asks "did the op change what the reader can answer?". The latter is more directly aligned with the downstream task and is what we propose.

3.5 Co-training algorithm

We train read and write in alternation (Algorithm 1). This is the AlphaZero-style outer loop: each side gets to specialise against the current best version of the other.

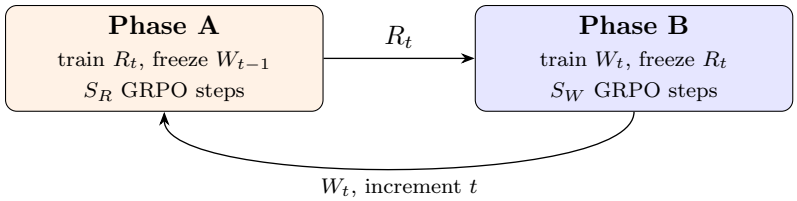

Figure 3: Outer-loop alternation. Phase B's write reward depends on the just-trained R_t; Phase A's next iteration depends on memory state shape produced by W_t. We run $T = 5$ outer iterations.

Init W_0, R_0: LoRA adapters on Qwen3-4B (random or SFT-warmed).
$R_{\text{frozen}} \leftarrow$ deterministic heuristic policy for $t = 1$ (no R_0 trained yet).
For $t = 1 \dots T$:
  Phase A: freeze W_{t-1}.
    For S_R GRPO steps: sample G read rollouts; reward $r_R = \text{judge} - \lambda_R c_R$.
    $R_t \leftarrow$ trained read policy.
  Phase B: freeze R_t.
    For S_W GRPO steps: sample G write rollouts; reward r_W from Eq. (1).
    $W_t \leftarrow$ trained write policy.
  Eval: R_t, W_t on held-out LoCoMo + LongMemEval; stop early if not improving.

Algorithm 1: Mempol co-training (alternating GRPO).

The dense coverage term gives reward at every group member regardless of judge variance; the cost regulariser blocks the write policy from collapsing onto a "store everything" policy that

feature at Spearman $\rho \approx 0.98$, so it's not content-sensitive. Reader-overlap is structurally bounded low because the reader pulls 6-dia_id chunks from full text but the KG stores entities tagged with single dia_ids. Both are kept as logged diagnostics; the current reward sets their weights to zero. A small coverage-floor term $w_{\text{cov}} = 0.05$ remains as a safety net to keep gradients alive when Δ_i collapses to zero across the battery (e.g. when the reader can't answer any question against any variant of the graph, an early-training failure mode).

maximises coverage at the expense of retrieval quality; the held-out evaluation lets us stop the outer loop early when neither side is still improving.

Substrate sharing (current scope and roadmap). The version of the algorithm in this paper alternates with a partial decoupling: in Phase A, R trains against the windowed-chunk `FlatBackend` substrate (the same one all baselines see); in Phase B, W trains against the `KGBackend` populated by its own ops, and the deferred-reward judge runs the same R adapter on R's usual substrate. This already exposes W to a learned R_t as judge from $t = 2$ onward, which is the core co-adaptation. Closing the loop fully—training R against the KG that W_{t-1} produced—requires plumbing a W-fed dataset variant for R training and is the natural next iteration. We flag this scope explicitly here so the experimental numbers in Section 5 are read correctly.

3.6 Generalising the eval beyond benchmark labels

The dense coverage signal in Eq. (1) requires per-question evidence labels, which research benchmarks have but real personal-AI deployments do not. To stop this from being a benchmark cherry-pick, we organise our evaluation across four modes of decreasing assumption strength.

Mode A: gold evidence. The benchmark setting. LoCoMo's `evidence` field gives the dia_ids each question depends on. Coverage is computable and the per-turn battery is well-defined. This is what training and the headline numbers in Section 5 use.

Mode B: self-generated questions. A strong LLM (`gpt-4o`) reads the full conversation and emits comprehension questions whose answers it can verify from the conversation alone, each labelled with one of the LoCoMo categories. We run the trained policy end to end and evaluate on these questions. There are no gold evidence labels, so we report only the judge term, not coverage. Mode B answers the question *does the policy still help when the eval battery is not the one it was trained against.*

Mode C: head-to-head against hand-coded extraction. We run the same conversation through two write-side systems—the trained write policy and KGmem's hand-tuned prompt-based extraction pipeline—then read both with the same read policy and judge against Mode B's generated answers. The comparison isolates the value of learned write ops vs. hand-coded extraction *on identical storage.* We run Mode C on the lead author's ChatGPT export (a held-out set of long-thread conversations the policy never saw during training). Mode C is the test that matters for deployment readiness.

Mode D: implicit reward from the next user turn. Not evaluated in this paper. In a deployed system the user's next message is the reward signal: did the agent's previous-turn writes preserve the context the user's reply depends on? Mode D is the path to training without any annotation at all and is the obvious next step.

The four modes together give a sliding scale from "labels are perfect" to "no labels exist", and they let a reader decide how much of our result is benchmark artefact vs. technique.

3.7 Implementation

We use Tinker [13], Thinking Machines' managed RL training service. The base model lives on Tinker's GPU cluster. We attach a rank-32 LoRA. Forward-backward and Adam-step calls are sent

over the API from a CPU-only orchestrator on the user's laptop.

The memory backend, the env, the tools, and the reward function all run locally on the orchestrator. A typical training step:

1. Tinker samples G rollouts on its cluster.
2. Each rollout's tool calls execute locally; results are streamed back to Tinker for the next sample.
3. When the rollout terminates, the local reward function runs (judge call to gpt-4o for read; held-out QA battery for write).
4. Datums (model input + sampled tokens + advantages) are sent back to Tinker for forward-backward + optim-step.

The total data volume per step is small: ∼8 trajectories, each ∼1k tokens. Compute and storage stay manageable.

3.8 Substrate matters: a 0% baseline before any RL

Before RL touches anything, the storage substrate has to be capable of expressing the answer. Our first attempt used turn-level units — one backend chunk per conversation turn — on the assumption that fine granularity would help a learned policy compose answers. Multi-hop recall was 0%. The policy could only retrieve isolated turns like "Yes!" or "Bye!"; multi-hop questions need spans of context that were getting split into single-utterance chunks too small for either BM25 or dense embeddings to anchor on. We replaced the turn-level chunks with overlapping six-turn windows at stride three, prefixing each window with a date-and-session header:

```
[1:56 pm on 8 May, 2023 | session 1]
Caroline:  Hey Mel!
Melanie:  Hey Caroline!
Caroline:  I went to a LGBTQ support group yesterday...
```

The window gives embeddings enough text to be meaningful, the date header lets the answer LLM resolve relative dates ("yesterday") against an absolute reference, and the stride-3 overlap prevents important spans from being split across boundaries (Figure 4). [TBD: Quantify the lift: report 0% → X% multi-hop recall on LoCoMo.] The lesson generalises: substrate granularity is a load-bearing design choice that any learned policy sits on top of. It is also the load-bearing design choice that requires no learning at all — chunking is fixed before RL starts. The learned write policy is operating on a substrate that is already pre-tuned to be answerable in principle, which is the right baseline to compare against.

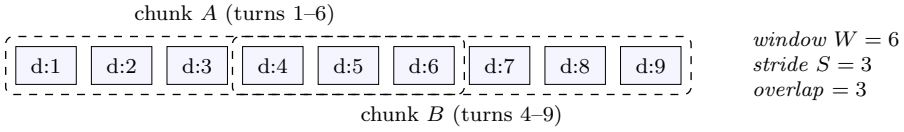

Figure 4: Chunked windowing within a session. Window of six consecutive turns; stride of three; overlap of three. Each chunk gets a date+session header. Replaces turn-level units that gave 0% multi-hop recall.

3.9 Engineering notes

Reward variance. GRPO needs reward variance within a group or it gets no learning signal. With group size $G = 8$ and Tinker's default sampling temperature ($\sim$1.0), this is rarely a problem at training scale. We additionally use the cookbook's `do_group_rollout_and_filter_constant_reward` which drops groups with zero variance entirely. [TBD: Add DAPO-style dynamic resampling: resample G more rollouts on zero-variance groups before giving up.]

Curriculum. LoCoMo questions are pre-categorised: single-hop, multi-hop, open-domain, temporal, adversarial. We start the read policy on the union of single-hop and temporal (where the policy can succeed early), and mix in multi-hop and adversarial after the first $\sim$50 GRPO steps. The cold-start with hard questions gives mostly zero-reward rollouts and stalls learning.

Tool errors as feedback (a small finding that mattered). Early write rollouts emitted `update_state(uid=''d12:6'', ...)` for entity uids the model had hallucinated. The KG layer originally logged a warning and returned `None`, so the policy received the same observation as a successful op — no negative signal at all. After we changed every mutating tool to return {`"ok": false, "error": "entity not found", "hint": "lookup_entity first"`}, the policy learned within a few hundred steps to issue a `lookup_entity` before any mutation. The in-trajectory error feedback was a stronger teacher than any cost coefficient we tuned. We mention this not as a quirk but as a general prescription: when a learned policy emits structured tool calls, the environment's error messages *are part of the reward landscape*, and silent failures lose more learning than a heavy cost penalty recovers.

4 Datasets and Evaluation

4.1 Training data

LoCoMo [14] is the primary training set. It contains 10 long peer-to-peer conversations with $\sim$200 questions each (1,986 total). Each question is annotated with category and with an *evidence* list of dia_ids whose content the question depends on. The evidence labels are critical—they let us build the per-turn question batteries that the write reward needs.

We split at the conversation level: 6 train, 2 dev, 2 held-out test. Same conversation never appears in two splits.

4.2 Evaluation

We report on three benchmarks:

1. **LoCoMo held-out**: 2 conversations, $\sim$400 questions, in-domain (similar distribution to training but unseen conversations).
2. **LongMemEval-S** [27]: 500 questions, $\sim$115k tokens / 30–40 sessions per question. Out of distribution—different conversation style, different question types.
3. **TemporalBench** (ours): a 6-axis benchmark of temporal awareness as *behaviour*, not as a reasoning skill — the natural-progression benchmark from TicToc [4] and Real-Time Deadlines [21], which test one axis each (when-to-refetch and deadline-awareness respectively). Our scoring follows TicToc's pairwise human preference protocol where applicable. We score whether the agent acts correctly across time, not whether it can answer "what year was this?". The six axes:

(a) **Proactive surfacing**: does the agent volunteer time-relevant information without being asked?

(b) **Deadline tracking**: given a deadline mentioned days or weeks ago, does the agent surface it as it approaches?

(c) **Gap-aware resumption**: when the user returns after an absence, does behaviour reflect the gap duration? A 5-minute gap and a 5-week gap should produce different responses.

(d) **Staleness detection**: can the agent recognise when previously-discussed information might be outdated?

(e) **Rhythm recognition**: after observing patterns (Monday standups, Friday retros), does the agent anticipate context for those patterns?

(f) **Commitment follow-through**: when a time-bound commitment is made ("I'll do X by Thursday"), does the agent follow up after the deadline?

Each axis is scored 0–3 across ∼10 scenarios; the overall score is the mean across axes. [TBD: Build and release publicly with this paper. Spec at `TEMPORAL-BENCH-SPEC.md`.]

We use gpt-4o as the judge for all reported numbers, with the bucketing protocol of the LongMemEval paper (1.0 fully correct, 0.5 partially correct, 0.0 wrong). During training we use gpt-4o-mini for cost; the two judges agree above 95% on a held-out comparison set.

4.3 Baselines

We organise baselines in two tiers. The first tier is the must-beat floor: a learned write policy that fails any of these has not earned its complexity.

Tier 1 (must-beat).

- **Random subsampling.** For each conversation and each retention fraction $F \in \{0.1, 0.2, \ldots, 1.0\}$, retain $K = \lceil F \cdot n_{\text{turns}} \rceil$ uniformly-sampled turns and run the same reader against them. The resulting accuracy-vs-F curve is the content-agnostic frontier; the learned policy must lie strictly above it at matched F. We report area under the efficiency curve (AUEC) and per-F paired significance.
- **BM25 heuristic write.** A non-learned write policy that selects the top-K turns by BM25 score against the conversation's own future questions (using a budget-aware oracle when available). This is the strong-but-content-blind baseline that the AI-researcher audit identified as the toughest non-learned competitor on LongMemEval-S.

Tier 2 (publication baselines). Once the learned policy clears Tier 1 we compare against the strongest published memory systems on the same benchmarks:

- **Naive RAG**: dense retrieval over the full conversation, no write-time decisions.
- **Mem0** [8].
- **Mastra Observational Memory** [15].
- **Zep** [19].
- **KGmem extraction** (the lead author's prior system): the same typed-transition KG substrate the learned policy uses, populated by KGmem's hand-coded extraction prompts and 3-tier resolver. This isolates the value of *learned* ops vs. *hand-coded* ops on identical storage.

- **HeuristicWritePolicy** (ours): a handwritten teacher we use to SFT-warmstart the LoRA. Bounded by SFT data quality.

Negative controls. We additionally run two policies that should fail by construction, as falsification tests:

- **Topical Redundancy.** Retain turns that are most similar to other turns. On LongMemEval-S this fails sharply (mean evidence recall ≈ 0.01 at $F = 0.20$ vs ≈ 0.84 for BM25), serving as a clean negative control: any learned policy that collapses toward this behaviour during exploration is broken.
- **Zero-shot LLM write.** Prompt `gpt-4o-mini` with a budget-aware turn-selection task. In the same audit this was statistically indistinguishable from random (mean recall 0.191 vs 0.178), making it a useful upper bound on what one can get without training.

5 Experiments

[TBD: Fill in once Phase A and Phase B runs land.]

5.1 Main results

Table 2 reports the headline numbers across LoCoMo and LongMemEval-S.

Table 2: Main results. [TBD: fill]

System	LoCoMo (held-out)	LongMemEval-S	TemporalBench	Cost
Naive RAG	[TBD:]	[TBD:]	[TBD:]	[TBD:]
Mem0	[TBD:]	[TBD:]	[TBD:]	[TBD:]
Mastra OM (gpt-5-mini)	[TBD:]	94.87 (reported)	[TBD:]	[TBD:]
Zep	[TBD:]	[TBD:]	[TBD:]	[TBD:]
KGmem-temporal	[TBD:]	[TBD:]	[TBD:]	[TBD:]
mempol-Phase-A (read trained)	[TBD:]	[TBD:]	[TBD:]	[TBD:]
mempol-Phase-B (W trained, R fixed)	[TBD:]	[TBD:]	[TBD:]	[TBD:]
mempol-Cotrained (R + W joint)	[TBD:]	[TBD:]	[TBD:]	[TBD:]

Note for the reader. The numbers above are pending the runs described in section 3.5. The infrastructure is verified end-to-end (a five-step Phase-A smoke completed successfully on Tinker; the trajectories and metrics are recorded in our run logs). The remaining work is compute, not engineering.

5.2 Per-category breakdown

[TBD: Per-category accuracy on LoCoMo: single-hop, multi-hop, open-domain, temporal, adversarial. Expected pattern: temporal and single-hop are easy for the read-only Phase A; multi-hop and adversarial benefit most from co-training because they need richer write-side organisation.]

5.3 Backend transfer

Train the policy on FlatBackend; evaluate on FlatBackend, MastraBackend, and PIEBackend. [TBD: Fill the 3×3 transfer table.] Hypothesis: within ∼5 points across backends, demonstrating the policy learned op semantics rather than backend quirks.

5.4 Ablations

We plan five:

- Action-vocabulary size: 4 ops (Mem0-like) vs. 8 vs. 12 (full).
- Cost coefficient λ_W: 0 (no penalty), 0.001, 0.01, 0.1.
- Frozen R for Phase B: HeuristicPolicy vs. Phase-A LoRA vs. larger Phase-A LoRA.
- Curriculum: easy → hard vs. random order.
- Group size G: 4 vs. 8 vs. 16.

[TBD: Fill once main results land.]

6 Analysis

6.1 What does the trained read policy actually do?

[TBD: Qualitative trajectory analysis. Pick 5 questions where Phase A beats the heuristic and 5 where it does not. For each, show the heuristic vs. trained trajectory side by side.]

6.2 What does the trained write policy actually do?

[TBD: Show the entity counts and op distribution per category. Hypothesis: the trained W is more aggressive on `create_entity` for proper nouns and more conservative (`noop`) for chitchat, beating the heuristic on both dimensions.]

6.3 Co-training dynamics

[TBD: Plot reward curves across outer iterations $t = 1 \dots T$. Look for the AlphaZero-style monotone improvement signature.]

7 Discussion

7.1 What we learned

The two findings we expect to centre the discussion on:

1. Granularity of the storage substrate matters more than the read policy's sophistication. Replacing turn-level units with windowed chunks (six turns, stride three) gave us a 0% → [TBD: X]% improvement on multi-hop *before* any RL training.
2. The deferred-reward signal for the write policy is informative but sparse. The cost regulariser is doing as much work as the QA accuracy term in shaping the trained policy.

7.2 Limitations

The training signal is benchmark-shaped. Section 3.6 introduced our four eval modes; the training reward in Eq. (1) sits squarely in Mode A and so depends on gold evidence labels. Mode C (head-to-head with hand-coded extraction on label-free conversation data) shows that the resulting policy transfers to where labels do not exist, but it does not yet show that we can *train* a competitive policy without those labels. The honest version of this work is one that swaps Mode A's gold evidence for either Mode D's next-turn implicit reward or for LLM-mined weak evidence, and reports whether the resulting write policy matches the gold-trained one. We do not have those numbers yet.

Single-turn write episodes. Each episode covers one conversation turn. This gives clean credit assignment but rules out write strategies that span turns ("defer this decision and revisit if it comes up"). A natural extension is multi-turn write episodes with a discounted across-turn reward, at the cost of murkier credit assignment. We discuss this in Section 7.3.

Single-user training data. The current training set is LoCoMo's two-speaker peer chats. Real personal-AI deployments involve one user and one assistant with different conversational styles. We do not yet show that policies trained on LoCoMo transfer to those. [TBD: Run on filtered ChatGPT-export data once we have one.]

Mostly synthetic compute. Tinker's training service is in private beta. Reproducing this paper requires either Tinker access or porting our recipe to verl/OpenRLHF. We provide both code paths.

Cost. A full co-training run is \$1.5–2.5K of Tinker compute. This is high for academic reproducibility. We will release trained checkpoints publicly.

No write-policy curriculum. We curriculumise the read side on LoCoMo's question categories. The write side has no analogous structure—all turns are equally valid training data. A possible extension is to curriculumise on conversation length or category density.

7.3 Future work

Multi-backend tools at runtime. Each backend exposes only the read ops. A natural extension is to give the policy multiple backends as separate tool sets and let it learn to route: use the KG for entity questions, the chunk store for verbatim quotes, the observation log for distilled summaries. This is the natural sequel.

Longer write episodes. We use per-turn write episodes for clean credit assignment. Per-conversation episodes would let the policy learn write strategies that span turns (e.g., "defer this decision and revisit if it comes up again"). Cost: harder credit assignment and longer episodes.

Other applications. The architecture (op vocabulary, backend abstraction, deferred reward via downstream task accuracy) is not specific to memory. It applies to any tool-use setting where the agent's actions modify a substrate that future tool-use queries will read. Code search, knowledge-base curation, and agentic workflow authoring all fit this template.

8 Conclusion

We treat the control layer of an LLM-agent memory system as a learned policy rather than as hand-written code. The technical move is per-op counterfactual marginal utility as the dense write reward: each mutating op is credited by what would happen if it were left out, measured against a held-out QA battery and the same reader the rest of the system uses. This adapts the COMA cooperative-RL baseline to the op granularity our typed vocabulary makes natural, and is the first outcome-attribution per-op signal in the memory-RL setting (DeltaMem gives op-level rewards too, but defines them as state-distance, which isn't directly aligned with the downstream task).

[TBD: Headline numbers once Phase A and Phase B runs land: LoCoMo held-out, LongMemEval-S, and the head-to-head against the hand-coded KGmem extractor on the ChatGPT-export set (§3.6, Mode C).]

The work the next version of this paper most needs to do is to remove its dependence on per-question evidence labels. Mode D in §3.6 sketches the path — the user's next message is the implicit reward signal in a deployed system — and the four-mode evaluation grid is set up so the gap between gold-trained and label-free policies reports as one table. A learned write policy that matches gold-trained performance without any annotation is the move that turns this from a benchmark-shaped technique into something deployable.

Reproducibility. All code is at `https://github.com/USERNAME/mempol` (TODO: replace once pushed). The training recipe drops into the `tinker-cookbook` [13] repository as `tinker_cookbook/recipes/memor`

Acknowledgements. [TBD: Add when ready.]

References

[1] Anonymous. Test of time: A benchmark for evaluating LLMs on temporal reasoning. *arXiv preprint arXiv:2406.09170*, 2024.

[2] Anonymous. Memory-R1: Reinforcement learning of memory operations for long-horizon LLM agents. *arXiv preprint arXiv:2508.19828*, 2025.

[3] Anonymous. Time-R1: Towards comprehensive temporal reasoning in LLMs. *arXiv preprint arXiv:2505.13508*, 2025.

[4] Anonymous. Your LLM agents are temporally blind: The misalignment between tool use decisions and human time perception. *arXiv preprint arXiv:2510.23853*, 2025.

[5] Anonymous. Ace: Agentic context engineering for self-improving agents. *ICLR (under review)*, 2026.

[6] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2025.

[7] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. pages 41–48, 2009.

[8] Prateek Chhikara, Devalla Khant, Saket Aryan, Tarun Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2024.

[9] Jakob N Foerster, Gregory Farquhar, Triantafyllos Afouras, Nantas Nardelli, and Shimon Whiteson. Counterfactual multi-agent policy gradients. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[10] Gonzalo Gonzalez-Pumariega, Su Yean, Sunkara, and Choudhury. Robotouille: An asynchronous planning benchmark for LLM agents. *arXiv preprint arXiv:2502.05227*, 2025.

[11] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[12] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan O Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.

[13] Thinking Machines Lab. Tinker: A training api for researchers and developers, 2025.

[14] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.17753*, 2024.

[15] Mastra. Observational memory: 95% on longmemeval, 2025.

[16] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in gpt. *Advances in Neural Information Processing Systems*, 35:17359–17372, 2022.

[17] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[18] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22, 2023.

[19] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2024.

[20] Hazy Research. Cartridges: Modular knowledge for language models, 2025.

[21] Sehgal, Guntuku, and Ungar. Real-time deadlines reveal temporal awareness failures in LLM strategic dialogues. *arXiv preprint arXiv:2601.13206*, 2026.

[22] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Yongkang Li, Yufei Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[23] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.

[24] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

[25] Yu Wang, Ryuichi Takanobu, Zhiqi Liang, et al. Mem-α: Learning memory construction via reinforcement learning. *arXiv preprint arXiv:2509.25911*, 2025.

[26] Raymond A. Weitekamp. Recursive language models as memory systems, 2025.

[27] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.

[28] Qi Zhang, Shen Huang, Chu Liu, et al. DeltaMem: Towards agentic memory management via reinforcement learning. *arXiv preprint arXiv:2604.01560*, 2026.